

Domain-Design Lern-Guide (Sales OS)

Vertiefung zu den 12 Interview-Fragen über das Domain-Modell (P1). Ziel: nicht die Antworten auswendig lernen, sondern das Denkmuster, mit dem man Struktur-Entscheidungen begründet — übertragbar auf jedes Software-Projekt.

Wie du diesen Guide benutzt

Lies pro Frage nicht nur die richtige Antwort, sondern die Analogie — die zeigt, dass es kein Sales-OS-Spezialwissen ist, sondern ein wiederkehrendes Muster (das dieselbe Entscheidung bei Datenbanken, Git, React, Stripe ... auftaucht). Am Ende steht eine Übungsmethode.

Die Meta-Lektion aus deinem Test (das Wichtigste zuerst)

Alle 6 Fehler waren dieselbe Falle: die Option „das Framework kann/verbietet X“.

- „Pydantic verbietet rekursive Modelle“ → falsch, es kann.
- „Pydantic erlaubt kein None auf Literal“ → falsch, `Literal[...] | None` geht.
- „Computed Properties gibt es in Pydantic nicht“ → falsch, `@computed_field`.
- „Validatoren sind der einzige Ort für abgeleitete Felder“ → falsch.
- „Pydantic kann Modelle nicht als Felder verschachteln“ → falsch.
- „Pydantic verlangt Strings für Audit-Felder“ → falsch.

Regel für die Zukunft: Wenn eine Erklärung lautet „die Sprache/das Framework zwingt uns dazu“, ist sie fast immer falsch. Moderne Frameworks können sehr viel. Eine Design-Entscheidung ist fast nie erzwungen — sie ist ein bewusster Trade-off: „Wir könnten X, wählen aber Y, weil Nutzen A den Preis B überwiegt.“

Das mentale Modell (bei jeder Struktur-Frage anwenden)

Vier Fragen, in dieser Reihenfolge:

1. Was wäre die naive Alternative? (verschachteln, computed, nullable, typisiert ...)
2. Kann das Tool beide? (meistens ja — also ist es kein Tool-Argument)
3. Was gewinnt die gewählte Variante, was kostet sie? (der eigentliche Trade-off)
4. Wann kippt die Entscheidung? (welche Zukunft macht die Alternative besser)

Wer Punkt 3 und 4 sauber benennt, klingt wie ein Senior Engineer. Wer bei Punkt 2 hängenbleibt („geht nicht anders“), klingt wie ein Junior.

Account

Q1 — Warum String-IDs statt Objekt-Verschachtelung?

Frage: Warum referenzieren Account/Contact/Deal einander über `account_id` (String), statt sich als Objekte zu verschachteln (z. B. `Account.contacts: list[Contact]`)?

Richtig: Flache ID-Referenzen halten die Aggregate unabhängig lad- und speicherbar und bilden normalisierte DB-Zeilen sauber ab. Preis: man muss Referenzen selbst auflösen (keine automatischen Joins).

Hintergrund: Das ist die uralte Entscheidung Normalisierung vs. Einbettung. Verschachtelt man Objekte, entsteht die Frage „wo ist die Wahrheit?“. Wenn derselbe Kontakt in zwei eingebetteten Bäumen steht, hast du zwei Kopien, die auseinanderlaufen. Flache Referenzen = eine Wahrheit pro Entität, alle anderen zeigen nur darauf.

Warum die Fallen falsch sind: Pydantic kann verschachteln und sogar rekursiv (Vorwärtsreferenzen). SQLite kann Beziehungen (Foreign Keys). Beides sind Tool-Argumente, und beide sind sachlich falsch.

Warum es wichtig ist: Verschachtelung erzeugt drei Probleme: (1) Duplikate/Sync-Bugs, (2) unklare Lade-Grenzen („lade ich den ganzen Baum bei jedem Zugriff?“), (3) Zyklen (Deal→Account→Deals→...), die Serialisierung und Speicherung sprengen. Die Repository-Schicht (P5) kann Aggregate nur deshalb einzeln laden/cachen/austauschen, weil sie flach referenziert sind.

Analogie aus der Software-Welt: Redux/Normalizr in der React-Welt. Die offizielle Empfehlung „Normalizing State Shape“ sagt exakt das: speichere Entitäten in einer flachen Map {id: entity} und referenziere per ID — nicht verschachtelt —, sonst bekommst du Sync-Bugs, wenn dieselbe Entität an zwei Stellen im Baum liegt. Dasselbe Prinzip: relationale DBs (Foreign Keys) vs. MongoDB-Embedding; GraphQL, das Referenzen lazy über Resolver auflöst.

Interview-Satz: „Ich referenziere per ID, damit jede Entität genau eine Quelle der Wahrheit hat und Aggregate unabhängig ladbar bleiben — den Preis, Joins selbst zu machen, nehme ich bewusst in Kauf.“

Q2 — Untypisiertes dict als Platzhalter

Frage: Wie ist `research_profile: dict | None` heute einzuordnen?

Richtig: Pragmatischer Platzhalter (Naht), der den M1-Typ nicht verfrüht baut — aber ein untypisiertes Loch ohne Validierung; nur vertretbar, weil temporär bis M1.

Hintergrund: Eine Seam (Naht) ist eine bewusst offen gelassene Stelle, an der später etwas Konkretes andockt. Die Kunst ist, die Naht zu markieren, ohne schon das ganze zukünftige Ding zu bauen (YAGNI) — aber im Bewusstsein, dass ein dict keine Validierung bietet (`extra="forbid"` greift nur eine Ebene, nicht in das dict hinein).

Warum die Fallen falsch sind: „dict ist endgültig besser“ ignoriert, dass du Typsicherheit verlierst. „Pflichtfeld“ würde jeden Account zwingen, ein Profil zu tragen, obwohl Recherche optional und später ist.

Warum es wichtig ist: Solche Nähte sind Schulden. Sie sind ok, wenn sie (a) klein, (b) markiert und (c) mit bekanntem Ablösetermin sind. Unkontrolliert werden sie zum „stringly-typed“-Sumpf (siehe Q11).

Analogie: TypeScript `unknown/any` als bewusster Escape-Hatch mit `// TODO: type this`; Protobuf `google.protobuf.Struct/Any` für noch-nicht-typisierte Felder; ein `jsonb`-Feld in Postgres, das man später in Spalten auflöst. Alle sagen: „hier ist absichtlich noch keine Struktur — temporär.“

Interview-Satz: „Das ist eine markierte Naht bis M1, kein Dauerzustand — ich akzeptiere die fehlende Validierung bewusst, weil die Migration zu einem typisierten ResearchProfile trivial ist.“

Contact

Q3 — UNBEKANNT-Default statt None oder Pflicht

Frage: Warum defaulten die Alignment-Felder auf UNKLAR/UNBEKANNT/KEIN_KONTAKT?

Richtig: Es kodiert „Abwesenheit von Beleg ≠ ein Wert“. Ein explizites UNBEKANNT verhindert, dass fehlende Info still als echte Einschätzung gilt, und hält jeden Kontakt valide, ohne Daten zu erfinden.

Hintergrund: Es gibt einen semantischen Unterschied zwischen drei Zuständen: „nicht gesetzt“ (None), „wir haben hingeschaut und wissen es nicht“ (UNBEKANNT), und „ist wirklich neutral“ (NEUTRAL). Ein explizites Unknown-Enum macht den mittleren Zustand first-class — statt ihn mit None oder einem Default-Wert zu verwechseln.

Warum die Fallen falsch sind: Pydantic erlaubt sehr wohl `Literal[...] | None`. Serialisierungs-Größe ist irrelevant. Beides sind Ablenkungen.

Warum es wichtig ist: In einem Anti-Halluzinations-Tool ist der Unterschied kritisch: Wenn ein Kontakt „NEUTRAL“ defaultet, liest der Analyzer später „diese Person ist neutral eingestellt“ — eine erfundene Behauptung. UNBEKANNT zwingt Ehrlichkeit. Zusätzlich sparst du None-Checks überall im Code (Null-Object-Idee).

Analogie: SQLs dreiwertige Logik (NULL ≠ leer ≠ 0) und die ewige Debatte „NULL vs. empty string“. JavaScript `undefined` vs. `null`. HTTP 204 No Content vs. 404 Not Found vs. 200 mit leerem Body — drei verschiedene „Nichts“. Das Null-Object-Pattern. Überall dieselbe Einsicht: „kein Wissen“ ist ein eigener, benennenswerter Zustand, kein `null`.

Interview-Satz: „Ich unterscheide bewusst ‚unbekannt‘ von ‚null‘ und von einem echten Wert — sonst behauptet das System Dinge, die nie in den Notes standen.“

Q4 — Alignment am Contact statt pro (Contact × Deal)

Frage: Warum wird Alignment am Contact zum Problem bei zwei Deals, warum trotzdem v1-ok?

Richtig: Dieselbe Person kann in Deal A CHAMPION, in Deal B BLOCKER sein; Alignment am Contact kollabiert das auf einen Wert und verliert die deal-spezifische Wahrheit. In v1 ok, weil dein Muster (meist 1 Deal/Account) die Kollision selten macht.

Hintergrund: Ein Attribut, das erst durch eine Beziehung entsteht, gehört auf die Beziehung, nicht auf eine der beiden Seiten. „Rolle im Deal“ ist eine Eigenschaft von (Person, Deal), nicht von Person. Das sauberste Modell wäre ein DealAlignment-Objekt, gekeyt auf (contact_id, deal_id) — ein Association Object.

Warum die Fallen falsch sind: „Rolle ist global“ ist sachlich falsch (der ganze Sinn von MEDDPICC ist deal-spezifische Politik). „Row-Count/Performance“ verfehlt das eigentliche — es ist ein Korrektheitsproblem, kein Performance-Problem.

Warum es wichtig ist: Das ist eine der häufigsten Modellierungs-Fehlerquellen: Attribute auf der falschen Seite einer M:N-Beziehung. Es zu kennen und bewusst zu akzeptieren („ich weiß, dass es bricht, aber mein Nutzungsmuster macht es selten“) ist genau die Senior-Denkweise — im Gegensatz dazu, es nicht zu sehen.

Analogie: GitHub: deine „Rolle“ (admin/write/read) ist keine Eigenschaft des User, sondern der Membership (User, Repository). Slack: dein Status/deine Rolle gilt pro Workspace, nicht global. SQLAlchemy Association Object: wenn die Verknüpfungstabelle eigene Spalten hat (role, since), wird sie ein eigenes Modell. Klassischer Fehler: „Gehalt“ auf Employee, obwohl jemand mehrere Verträge hat.

Interview-Satz: „Rolle-im-Deal ist ein Beziehungs-Attribut; korrekt gehört es auf (Contact, Deal). Für v1 lege ich es auf den Contact, weil ich meist ein Deal pro Account habe — und dokumentiere das als bewusste, refactorbare Schuld.“

Deal

Q5 — Abgeleiteten Wert materialisieren statt computed

Frage: Warum wird `win_probability` als echtes Feld gespeichert, nicht beim Lesen berechnet?

Richtig: Materialisieren macht den Wert überschreibbar (ein Rep kann vom Stage-Default abweichen) und abfragbar in der DB; eine reine computed property könnte keinen manuellen Override halten. Preis: kann von der Stage abdriften.

Hintergrund: „Abgeleitet“ heißt nicht automatisch „computed on read“. Sobald der abgeleitete Wert (a) übersteuert werden können muss oder (b) so, wie er damals war, festgehalten werden muss, speicherst du ihn. Der Validator füllt nur, wenn nichts gesetzt ist (`mode="after", if None`) — so bleibt der Override erhalten.

Warum die Fallen falsch sind: Pydantic hat `@computed_field/@property`. Performance ist bei einem Single-User-Tool kein Argument. Beides Ablenkung.

Warum es wichtig ist: Der Unterschied „computed vs. stored“ ist eine ständige Praxis-Entscheidung. Faustregel: `computed`, wenn der Wert immer eine reine Funktion aktueller Daten ist; `stored`, wenn er übersteuerbar sein oder einen historischen Zeitpunkt einfrieren muss. Der Preis von `stored` ist Drift — deshalb braucht es eine klare Regel, wann neu abgeleitet wird.

Analogie: Eine Bestellung speichert den `price` zum Kaufzeitpunkt, statt ihn aus dem aktuellen Produktpreis zu berechnen — sonst ändert sich die Historie, wenn der Preis morgen steigt. Stripe speichert `amount` auf der Invoice. SQL: `GENERATED ALWAYS AS ... STORED` (materialisiert) vs. `VIRTUAL` (computed) — die Sprache bietet bewusst beide, weil es ein echter Trade-off ist. Ein `display_name`, der aus Vor+Nachname defaultet, aber überschreibbar ist.

Interview-Satz: „Ich materialisiere, weil der Wert überschreibbar und abfragbar sein muss — computed könnte den manuellen Override nicht halten. Den Preis (Drift zur Stage) kontrolliere ich, indem der Default nur greift, wenn nichts gesetzt ist.“

Q6 — Import von `settings` in die Domain

Frage: Verletzt `from settings import STAGE_GATES` die Regel „Domain hat keine Cross-Layer-Imports“?

Richtig: Nein — `settings` ist ausdrücklich Querschnitt („keine eigene Schicht, von überall nutzbar“), kein Layer. Der Import hält `STAGE_GATES` als einzige Quelle der Wahrheit (DRY), statt `Win-%` in die Domain zu duplizieren.

Hintergrund: Die „Dependency Rule“ (Clean Architecture) verbietet, dass innere Schichten von äußeren abhängen. Aber Querschnittsbelange (Konfiguration, Konstanten, Logging) sind eine anerkannte Ausnahme: Sie hängen selbst von nichts ab (sie sind ein „Sink“/Blatt im Abhängigkeitsgraphen), also erzeugt ihr Import nie einen Zyklus und nie eine Kopplung an Businesslogik.

Warum die Fallen falsch sind: „Harte Verletzung → hardcoden“ würde `STAGE_GATES` duplizieren (DRY-Bruch, zwei Wahrheiten). „Egal, Python erzwingt Architektur eh nicht“ ist der zynische Junior-Reflex — Architektur ist Disziplin, nicht Compiler-Zwang.

Warum es wichtig ist: Man muss die Dependency Rule und ihre legitimen Ausnahmen kennen. `Config/Constants` als Blatt-Modul, das jeder importieren darf, ist ein Standard-Muster. Der Test: Importiert das Modul selbst irgendetwas aus einer Schicht? Nein → sicher als Querschnitt.

Analogie: Shared Kernel in DDD. Ein `constants-` oder `config-`Modul ohne eigene Abhängigkeiten, das überall importiert werden darf. Logging-Frameworks (`slf4j`, `Pythons logging`) werden überall importiert, ohne dass das als Architektur-Bruch gilt. In Clean Architecture heißen solche Dinge „Entities/Cross-cutting“ und liegen außerhalb der Ringe.

Interview-Satz: „`settings` ist ein abhängigkeitsfreies Querschnitts-Blatt, kein Layer — es zu importieren erzeugt weder Zyklus noch Businesskopplung, und ich halte `STAGE_GATES` DRY an einer Stelle.“

Activity

Q7 — Hash in der Domain, aber Normalisierung erst in Ingestion

Frage: Warum ein (nicht-normalisierter) SHA-256-Hash in der Domain, wenn Normalisierung Ingestion-Sache (P6) ist?

Richtig: Die Domain liefert einen billigen Default-Content-Hash, damit eine nackte Activity in sich konsistent und testbar ist; Ingestion kann einen normalisierten Hash explizit übergeben. Die Domain normalisiert bewusst nicht — das bleibt P6 — also leckt keine Ingestion-Logik in die Domain.

Hintergrund: Trennung von Zuständigkeiten mit einer sinnvollen Default-Grenze: Die Domain garantiert die Invariante „hash == f(text)“ für ein isoliert erzeugtes Objekt. Die Policy (was „gleich“ bedeutet: Groß/Klein, Whitespace, Unicode) gehört in die Ingestion, weil sie fachlich ist und sich ändern kann.

Warum die Fallen falsch sind: „Der Domain-Hash IST die Idempotenz“ ist gefährlich falsch — echte Idempotenz braucht Normalisierung. „Validatoren sind der einzige Ort für abgeleitete Felder“ ist ein Tool-Mythos (Factories, Repository, computed gehen auch).

Warum es wichtig ist (echter Bug-Riecher!): Genau hier lauert ein Bug: Wenn P6 normalisiert und den Hash setzt, aber irgendein Codepfad den Domain-Default nimmt, existieren zwei Hash-Regime — und Dedup findet Duplikate nicht mehr. Das muss in P6 diszipliniert werden (immer normalisieren, bevor die Activity gebaut wird). Ein Senior sieht diese Nahtstelle vorher.

Analogie: Unicode-Normalisierung (NFC) vor Vergleich/Hash — der klassische Bug, dass zwei „identische“ Strings unterschiedlich hashen, weil einer nicht normalisiert ist (z. B. é als ein Codepoint vs. e + Accent). Exakt das „zwei Regime“-Problem. Git hasht Blobs, behandelt aber Zeilenenden (autocrlf) als separate Policy — Mismatch erzeugt Phantom-Diffs. Passwort-Hashing: der Hash gehört zum User, die Salt/Pepper-Policy zur Security-Schicht.

Interview-Satz: „Die Domain gibt einen Default-Hash für Selbstkonsistenz, aber die Normalisierungs-Policy bleibt in der Ingestion — und ich markiere das als Stelle, wo zwei Hash-Regime entstehen könnten, wenn man nicht diszipliniert normalisiert.“

Q8 — Append-only (Activity) vs. mutable (Deal/Contact)

Frage: Struktureller Grund für den Unterschied?

Richtig: Activities sind unveränderliche historische Fakten (ein Call fand statt, sein Text war, wie er war) — Historie umzuschreiben ist sinnlos/gefährlich. Deal/Contact sind aktueller Zustand, der sich legitim ändert (updated_at). Snapshots/Corrections folgen demselben Split.

Hintergrund: Das ist die Event-vs-State-Unterscheidung (Event-Sourcing-light). Events sind Dinge, die passiert sind — sie sind per Definition endgültig. State ist die aktuelle Sicht, ein Ergebnis der Events. State darf man überschreiben, Events nie.

Warum die Fallen falsch sind: „Schreib-Kosten sparen“ und „weniger Felder“ sind oberflächlich — es geht um die Natur der Daten (Faktum vs. Momentaufnahme), nicht um Speicher.

Warum es wichtig ist: Das ist das organisierende Prinzip des ganzen Modells. Nur weil Activities und Snapshots append-only sind, kannst du überhaupt Trend, Historie, Won/Lost-Analyse und Debugging rekonstruieren. Wer bei jedem neuen Modell fragt „ist das ein Event oder State?“, modelliert automatisch sauber.

Analogie: Git: Commits sind unveränderlich (Event-Log), das Working Tree ist mutabel (State). Buchhaltung: eine gebuchte Zeile korrigiert man nie durch Edit, sondern durch eine Gegenbuchung (append) — exakt unsere „neuer Snapshot statt Überschreiben“-Regel. Kafka: der Log ist append-only, die KTable/Materialized View ist der aktuelle Zustand. Bank: einzelne Transaktionen (Events) vs. Kontostand (State).

Interview-Satz: „Ich trenne Events von State: was passiert ist, ist append-only und unveränderlich; was den aktuellen Zustand beschreibt, ist mutabel. Daraus fällt Historie, Trend und Auditierbarkeit gratis heraus.“

MEDDPICC

Q9 — `dict[str, DimensionAssessment]` vs. 8 benannte Felder

Frage: Welcher Trade-off?

Richtig: Du gibst statische Typsicherheit/Autocomplete je Dimension auf (ein vertippter Key fällt erst zur Laufzeit im Validator auf) und gewinnst Flexibilität: MEDDICC vs. MEDDPICC unterscheiden sich im Dimensions-Set (`paper_process`), Teil-Snapshots sind natürlich, das Iterieren ist uniform.

Hintergrund: Offenes vs. geschlossenes Schema. Ein festes Modell mit 8 Feldern ist geschlossen und typsicher; eine Map ist offen und flexibel. Beides ist legitim — die Wahl hängt davon ab, ob das Set variiert (dann Map) oder fix ist (dann Felder).

Warum die Fallen falsch sind: „strikt besser“ ist zu absolut — benannte Felder hätten echte Vorteile (IDE, mypy). „Pydantic kann nicht verschachteln“ ist schlicht falsch (ein 8-Felder-Modell mit `DimensionAssessment` je Feld wäre trivial).

Warum es wichtig ist: Das ist eine echte Streitfrage — und die reifste Antwort im Interview räumt das ein: „Ich habe die Map gewählt, weil das Dimensions-Set zwischen MEDDICC/MEDDPICC variiert und Teil-Snapshots natürlich sein sollen; ein Staff Engineer könnte hier mit gutem Grund die 8 typisierten Felder bevorzugen.“ Nuance schlägt Dogmatismus.

Analogie: HTTP-Header als `Map<String,String>` (flexibel, stringly-typed) vs. ein typisiertes Request-Objekt. Postgres `jsonb`-Spalte vs. typisierte Spalten. Protobuf `map<string, X>` vs. explizite Felder. React: `props` als offenes Objekt vs. strikte `PropTypes`/TS-Interfaces. Immer dieselbe Achse: Erweiterbarkeit vs. Compile-Time-Sicherheit.

Interview-Satz: „Map, weil das Dimensions-Set variiert und Teil-Snapshots natürlich sein sollen — ich zahle mit Laufzeit- statt Compile-Zeit-Prüfung; die 8-Felder-Variante wäre eine legitime, typsicherere Alternative.“

Q10 — `trend` einfrieren statt on-the-fly diffen

Frage: Warum sitzt `trend` als Feld im Snapshot, statt beim Vergleich zweier Snapshots berechnet zu werden?

Richtig: Der Trend ist das Urteil des Analyzers zum Zeitpunkt des Snapshots (welcher Vorgänger, was gewichtet wurde), kein trivial neu ableitbarer Diff. Einfrieren macht den Snapshot zu einem in sich abgeschlossenen, append-only Record, der korrekt bleibt, auch wenn später weitere Snapshots kommen. Preis: Redundanz.

Hintergrund: Man speichert ein berechnetes Ergebnis, wenn (a) die Berechnung nicht rein reproduzierbar ist (ein LLM-Urteil mit Kontext) und (b) das Ergebnis zu einem Zeitpunkt gehört, der eingefroren gehört. Das ist kein simpler Diff zweier Zahlen, sondern eine Bewertung mit Begründung.

Warum die Fallen falsch sind: „reine Denormalisierung, entfernen“ erkennt, dass das Urteil nicht reproduzierbar ist. „Snapshots sind nicht geordnet“ ist falsch — sie haben `created_at` und `source_activity_ids`.

Warum es wichtig ist: „Judgment freezing“ ist ein wichtiges Muster: Wenn ein berechneter Wert von veränderlicher Logik oder menschlichem/Modell-Urteil abhängt, speicherst du das Ergebnis samt Kontext, statt es später neu zu erraten. Sonst ändert sich „die Vergangenheit“, wenn sich die Logik ändert.

Analogie: Ein Kredit-Score zum Entscheidungszeitpunkt wird gespeichert, nicht neu berechnet (die Scoring-Formel ändert sich). Der FX-Kurs, mit dem eine Transaktion umgerechnet wurde, wird eingefroren. Eine ML-Prediction wird mit `model_version` gespeichert. Ein Changelog/Release-Note schreibt man beim Release, weil es menschliches Urteil einfängt, das ein `git diff` nie hätte.

Interview-Satz: „`trend` ist ein eingefrorenes Analyzer-Urteil mit Kontext, kein Zahlen-Diff — ich speichere es, damit der Snapshot ein selbstständiger historischer Record bleibt, auch wenn die Bewertungslogik sich später ändert.“

Correction

Q11 — `original_value/corrected_value` als `str`

Frage: Warum alles zu String serialisieren?

Richtig: Bewusste Vereinfachung: ein uniformer String vermeidet, jeden möglichen korrigierten Typ (Enum/Liste/verschachtelt) in v1 zu modellieren — um den Preis fehlender Typvalidierung und Struktur (man kann nicht prüfen, ob eine korrigierte Confidence ein legaler Enum-Wert ist). Vertretbar, weil Corrections bis nach M4 nur gesammelt, nicht injiziert werden.

Hintergrund: Ein generischer Audit-/Änderungs-Record steht vor der Wahl: typisiert (strukturiert, validierbar, aber pro Typ Aufwand) oder stringly-typed (uniform, billig, aber blind). Solange die Daten nur abgelegt und vom Menschen gelesen werden, reicht String. Sobald sie maschinell zurückgespielt werden, will man Struktur.

Warum die Fallen falsch sind: „immer kurzer Text“ ist eine unbelegte Annahme. „Pydantic verlangt Strings“ ist frei erfunden (Union/Any/JSON gingen).

Warum es wichtig ist: Das ist Primitive Obsession (Fowler) — ein bekanntes Anti-Pattern, das hier bewusst und temporär akzeptiert wird. Der Reifegrad zeigt sich darin, die Schuld zu benennen und ihren Ablösepunkt (Feedback-Injektion nach M4) zu kennen.

Analogie: Die klassische generische Audit-Tabelle `audit_log(old_value TEXT, new_value TEXT)` — überall anzutreffen, mit genau dieser bekannten Grenze (nicht nach

Typ abfragbar/validierbar). Django `LogEntry` speichert eine JSON-Change-Message.

„Stringly typed“ allgemein: Zahlen/Enums als Strings durchreichen und die Typsicherheit an der Grenze verlieren.

Interview-Satz: „String ist Primitive Obsession, die ich für die reine Sammelphase bewusst in Kauf nehme — sobald Corrections maschinell injiziert werden (nach M4), braucht `value` einen typisierten/JSON-Wert.“

Q12 — `field_path` als Freitext-String

Frage: Welche Fehlerklasse lädt das ein?

Richtig: Keine referenzielle Integrität: Tippfehler oder umbenannte Felder (`dimensions.champ.confidence`) werden beim Schreiben nicht erkannt und zeigen still ins Leere, sodass spätere Injektion/Replay das Ziel nicht zuverlässig findet. Strukturierte Referenzen (`snapshot_id` + Dimension + Attribut) wären validierbar.

Hintergrund: Eine stringly-typed Referenz ist ein Zeiger ohne Prüfung. Der Compiler/das Schema weiß nicht, dass der String auf ein echtes Feld zeigen soll — also fällt kein Fehler an, wenn er ins Nichts zeigt. Strukturierte Referenzen machen die Zielmenge explizit und damit validierbar.

Warum die Fallen falsch sind: „vollständig sicher“ ist das Gegenteil der Wahrheit. „Speicherverbrauch“ verfehlt das Problem komplett (es geht um Korrektheit, nicht Bytes).

Warum es wichtig ist: Koppelt direkt an Q11 — stringly-typed value + stringly-typed path = ein Audit-Record, den man weder validieren noch sicher zurückspielen kann. Für v1 (nur sammeln) ok; für den Feedback-Loop (M4+) muss beides härter werden. Das Muster „Magic Strings statt typisierter Referenzen“ ist eine der häufigsten stillen Fehlerquellen in Software.

Analogie: React String-Refs (`ref="myInput"`) wurden u. a. deshalb deprecated — keine Prüfung, brüchig — zugunsten von `useRef`/Callback-Refs. `i18n-Translation-Keys` als Strings (`"home.title"`), die still „verschwinden“, wenn man sie umbenennt. DB-Spalten per String-Name ansprechen vs. typisiertes ORM-Column-Objekt (Rename bricht den String lautlos). CSS-Selektoren/JSONPath als Magic Strings.

Interview-Satz: „Ein Freitext-Pfad ist ein ungeprüfter Zeiger — Tippfehler zeigen lautlos ins Leere. Für die Sammelphase ok, aber fürs Zurückspielen brauche ich strukturierte, validierbare Referenzen.“

Übungsmethode (so wirst du besser)

1. Nimm jede Frage und beantworte sie zuerst mit dem 4-Punkte-Modell (Alternative? Kann das Tool beides? Gewinn/Kosten? Wann kippt es?) — bevor du die Lösung liest.
2. Rot markieren: Jede Erklärung, die „das Tool zwingt/verbietet“ enthält, ist verdächtig. Prüfe, ob das Tool es wirklich nicht kann (meist doch).
3. Formuliere laut den Interview-Satz — er hat immer die Form „Ich wähle Y statt X, weil Nutzen A > Preis B; X wäre besser, wenn Zukunft Z.“

4. Sammle eigene Analogien: Wenn du eine Entscheidung im Code triffst, frag „wo habe ich dieses Muster schon gesehen?“ (Git, SQL, React, Stripe, Kafka ...). Die Wiedererkennung ist das, was Senior-Antworten souverän macht.
5. Die 6 Muster aus diesem Guide, die überall wiederkommen:
Normalisierung vs. Einbettung · Event vs. State · computed vs. stored ·
offenes vs. geschlossenes Schema · Beziehungs-Attribut gehört auf die Beziehung ·
stringly-typed vs. typisiert (referenzielle Integrität).